

Práctica: Semilla de datos y consultas LINQ de alto rendimiento

Paso a paso — Poblar roles maestros con HasData, auditar cambios y optimizar la API REST con Include, AsNoTracking y Select

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Configurar datos maestros semilla (Seed Data) en EF Core para Roles y Centros de Costo mediante HasData() y aplicar la migración sobre SQL Server.
- Sobrescribir SaveChangesAsync en SolicitudesDbContext para poblar automáticamente los campos de auditoría (FechaCreacion, CreadoPor, etc.) en toda EntidadAuditable.
- Conectar SolicitudService a SolicitudesDbContext, aplicando Eager Loading con Include y ThenInclude para eliminar el antipatrón del N+1 en el endpoint por ID.
- Optimizar las consultas de solo lectura en el endpoint GET de solicitudes mediante AsNoTracking().
- Construir un endpoint con proyección tipada usando Select() para transferir únicamente las columnas requeridas por el cliente en un DTO ligero.
- Configurar el registro del servicio con ciclo de vida Scoped en Program.cs y verificar los endpoints REST en ejecución.

2. Desarrollo paso a paso

Paso 1 — Configurar datos semilla con HasData() en RolConfiguration y CentroCostoConfiguration

Para asegurar que el sistema inicie con los roles y centros de costo requeridos por las reglas del negocio, definimos los datos maestros iniciales usando HasData(). Configuramos cada entidad en su

propio archivo dentro de SolicitudesApp.Infraestructura/Persistencia con el código completo a continuación:

Archivo 1: SolicitudesApp.Infraestructura/Persistencia/RolConfiguration.cs — Define la clave primaria, la longitud máxima del nombre y los 4 roles del sistema (Usuario, Revisor, Aprobador, Tesorería):

RolConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infraestructura.Persistencia
{
    public class RolConfiguration : IEntityTypeConfiguration<Rol>
    {
        public void Configure(EntityTypeBuilder<Rol> builder)
        {
            builder.HasKey(r => r.Id);
            builder.Property(r => r.Nombre).IsRequired().HasMaxLength(50);

            builder.HasData(
                new Rol { Id = 1, Nombre = "Usuario" },
                new Rol { Id = 2, Nombre = "Revisor" },
                new Rol { Id = 3, Nombre = "Aprobador" },
                new Rol { Id = 4, Nombre = "Tesoreria" }
            );
        }
    }
}
```

Archivo 2: SolicitudesApp.Infraestructura/Persistencia/CentroCostoConfiguration.cs — Mapea la tabla CentrosCosto, sus propiedades y los 3 centros de costo iniciales de la organización:

CentroCostoConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infrastructure.Persistencia
{
    public class CentroCostoConfiguration : IEntityTypeConfiguration<CentroCosto>
    {
        public void Configure(EntityTypeBuilder<CentroCosto> builder)
        {
            builder.ToTable("CentrosCosto");
            builder.HasKey(c => c.Id);
            builder.Property(c => c.Codigo).IsRequired().HasMaxLength(20);
            builder.Property(c => c.Nombre).IsRequired().HasMaxLength(100);

            builder.HasData(
                new { Id = 1, Codigo = "CC-TI-01", Nombre = "Tecnologías de la Información",
                    Activo = true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" },
                new { Id = 2, Codigo = "CC-OPS-02", Nombre = "Operaciones y Logística", Activo =
                    true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" },
                new { Id = 3, Codigo = "CC-VEN-03", Nombre = "Ventas y Marketing", Activo = true,
                    FechaCreacion = new DateTime(2026, 1, 1, 0, 0, 0, DateTimeKind.Utc),
                    CreadoPor = "seed" }
            );
        }
    }
}
```

Claves primarias explícitas en HasData: EF Core necesita que especifiques el Id en HasData() para determinar si el registro ya existe en la base de datos o si debe generarse un INSERT durante la migración.

Paso 2 — Crear y aplicar la migración en Windows (PowerShell)

Abre una terminal PowerShell en Windows posicionada en la raíz de la solución (C:\Users\...\SolicitudesApp). Dado que las carpetas de los proyectos están directamente en la raíz de la solución (sin subcarpeta src/), ejecuta los comandos indicando directamente los nombres de los proyectos:

PowerShell (Windows) – Ejecutar en la raíz de la solución SolicitudesApp

```
# 1. Crear la migración para los datos semilla:
dotnet ef migrations add MigracionSemilla --project SolicitudesApp.Infrastructure --startup-project SolicitudesApp.Api

# 2. Aplicar la migración a la base de datos SQL Server:
dotnet ef database update --project SolicitudesApp.Infrastructure --startup-project SolicitudesApp.Api
```

Solución al error 'Unable to retrieve project metadata': Si recibes el error 'Unable to retrieve project metadata. Ensure it's an SDK-style project', la causa es que dotnet ef no encuentra el proyecto porque la ruta especificada no existe (por ejemplo, usar 'src/SolicitudesApp.Infrastructure' cuando no existe una carpeta 'src/'). En Windows PowerShell, ubícate en la raíz de la solución (donde está el archivo .sln o .slnx) y referencia directamente 'SolicitudesApp.Infrastructure' y 'SolicitudesApp.Api' (o './SolicitudesApp.Infrastructure' / '..\SolicitudesApp.Infrastructure').

Paso 3 — Automatizar la auditoría en SaveChangesAsync

Sobrescribimos SaveChangesAsync en SolicitudesDbContext para que toda entidad que herede de EntidadAuditable reciba su fecha UTC automáticamente al ser persistida, evitando asignaciones manuales en los controladores.

SolicitudesDbContext.cs

```
public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
{
    var entradas = ChangeTracker.Entries<EntidadAuditable>();

    foreach (var entrada in entradas)
    {
        if (entrada.State == EntityState.Added)
        {
            entrada.Entity.FechaCreacion = DateTime.UtcNow;
            if (string.IsNullOrEmpty(entrada.Entity.CreadoPor))
                entrada.Entity.CreadoPor = "sistema"; // Se vinculará al usuario autenticado en
S13
        }
        else if (entrada.State == EntityState.Modified)
        {
            entrada.Entity.FechaModificacion = DateTime.UtcNow;
            if (string.IsNullOrEmpty(entrada.Entity.ModificadoPor))
                entrada.Entity.ModificadoPor = "sistema";
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}
```

Auditoría centralizada con ChangeTracker: ChangeTracker.Entries<T>() intercepta todas las entidades modificadas en la transacción actual, asegurando marcas de tiempo homogéneas en UTC y garantizando trazabilidad contable.

Paso 4 — Conectar SolicitudService y Eager Loading (Include)

Reemplazamos los datos estáticos de SolicitudService inyectando SolicitudesDbContext. Implementamos ObtenerPorIdAsync cargando la solicitud junto a su solicitante, centro de costo, detalles y aprobaciones en un único viaje a la BD mediante JOINS explícitos.

Application/Solicitudes/SolicitudService.cs y SolicitudesController.cs

```
// Application/Solicitudes/SolicitudService.cs
public class SolicitudService(SolicitudesDbContext db, AsignadorAprobador asignador) :
ISolicitudService
{
    public async Task<SolicitudRespuestaDto?> ObtenerPorIdAsync(int id)
    {
        // Eager Loading: genera JOINS explícitos en lugar de múltiples consultas separadas
        var solicitud = await db.Solicitudes
            .Include(s => s.Solicitante)
            .Include(s => s.CentroCosto)
            .Include(s => s.Detalles)
            .Include(s => s.Aprobaciones)
            .ThenInclude(a => a.Responsable)
            .FirstOrDefaultAsync(s => s.Id == id);

        return solicitud is not null ? Mapear(solicitud) : null;
    }
}

// Api/Controllers/SolicitudesController.cs: endpoint GET por ID asíncrono
[HttpGet("{id:int}")]
public async Task<IActionResult> ObtenerPorId(int id)
{
    var solicitud = await service.ObtenerPorIdAsync(id);
    return solicitud is not null ? Ok(solicitud) : NotFound(new { mensaje = $"Solicitud #{id} no existe" });
}
```

El peligro del antipatrón N+1: sin Include, consultar 100 solicitudes e iterar sus detalles dispararía 101 consultas SQL independientes (1 + 100). Include() genera un único JOIN relacional que resuelve la jerarquía en un solo viaje de red.

Paso 5 — Optimización de solo lectura con AsNoTracking() en la API REST

Para listar solicitudes en un endpoint GET masivo, apagamos el Change Tracker de EF Core con AsNoTracking() para ahorrar memoria RAM y tiempo de CPU, devolviendo las solicitudes ordenadas por fecha reciente.

Application/Solicitudes/SolicitudService.cs y SolicitudesController.cs

```
// Application/Solicitudes/SolicitudService.cs
public async Task<IReadOnlyList<SolicitudLecturaDto>> ListarLecturaAsync(EstadoSolicitud?
filtroExcluir = null)
{
    // AsNoTracking: hasta un 60% más rápido para consultas de solo lectura
    var consulta = db.Solicitudes
        .AsNoTracking()
        .Include(s => s.Solicitante)
        .Include(s => s.Detalles)
        .AsQueryable();

    if (filtroExcluir.HasValue)
        consulta = consulta.Where(s => s.Estado != filtroExcluir.Value);

    return await consulta
        .OrderByDescending(s => s.FechaCreacion)
        .Select(s => new SolicitudLecturaDto(s.Id, s.Motivo, s.MontoTotal, s.Estado.ToString(),
s.Solicitante!.Nombre, s.FechaCreacion, s.Detalles.Count))
        .ToListAsync();
}

// Api/Controllers/SolicitudesController.cs: endpoint GET general optimizado
[HttpGet]
public async Task<IActionResult> ListarLectura([FromQuery] EstadoSolicitud? excluirEstado = null)
{
    var listado = await service.ListarLecturaAsync(excluirEstado);
    return Ok(listado);
}
```

Rendimiento con AsNoTracking: úsalo siempre en consultas de lectura donde no se invoque SaveChangesAsync(). Evita que EF Core mantenga copias de comparación de estado en memoria, reduciendo la presión sobre el Garbage Collector.

Paso 6 — Proyección con Select() en el endpoint de resumen

Definimos un DTO ligero y proyectamos con Select() para generar un SELECT en SQL únicamente con las columnas requeridas, transfiriendo solo los datos indispensables para vistas de resumen o tablas.

Dtos.cs, SolicitudService.cs y SolicitudesController.cs

```
// Application/Solicitudes/Dtos.cs: DTO proyectado ligero
public record SolicitudResumenDto(int Id, string Motivo, decimal MontoTotal, string Solicitante,
int TotalItems, string Estado);

// Application/Solicitudes/SolicitudService.cs
public async Task<IReadOnlyList<SolicitudResumenDto>> ListarResumenAsync(EstadoSolicitud? estado
= null)
{
    var consulta = db.Solicitudes.AsNoTracking();
    if (estado.HasValue)
        consulta = consulta.Where(s => s.Estado == estado.Value);

    return await consulta
        .Select(s => new SolicitudResumenDto(
            s.Id, s.Motivo, s.MontoTotal,
            s.Solicitante != null ? s.Solicitante.Nombre : "",
            s.Detalles.Count, s.Estado.ToString()))
        .ToListAsync();
}

// Api/Controllers/SolicitudesController.cs: endpoint GET /api/solicitudes/resumen
[HttpGet("resumen")]
public async Task<IActionResult> ListarResumen([FromQuery] EstadoSolicitud? estado = null)
{
    var resumen = await service.ListarResumenAsync(estado);
    return Ok(resumen);
}
```

Proyección a nivel de base de datos: a diferencia de mapear objetos en memoria con LINQ to Objects, `Select()` sobre `IQueryable` se traduce directamente a columnas SQL (`SELECT s.Id, s.Motivo, ...`), ahorrando ancho de banda de red y tiempo de deserialización.

Paso 7 — Registro Scoped en Program.cs y prueba de la API REST

Actualizamos el registro de dependencias en `Program.cs` configurando `ISolicitudService` con ciclo de vida Scoped (ya que consume `SolicitudesDbContext`) y probamos los endpoints desde Swagger o cliente HTTP.

Program.cs y SolicitudesApp.Api.http

```
// Api/Program.cs: registro Scoped en el contenedor de dependencias
builder.Services.AddScoped<ISolicitudService, SolicitudService>();

// SolicitudesApp.Api.http: pruebas de los endpoints de la API REST
### 1. Obtener solicitud con Eager Loading (Include)
GET http://localhost:5072/api/solicitudes/1
Accept: application/json

### 2. Listado de solo lectura optimizado (AsNoTracking)
GET http://localhost:5072/api/solicitudes?excluirEstado=Pagada
Accept: application/json

### 3. Listado proyectado con Select()
GET http://localhost:5072/api/solicitudes/resumen
Accept: application/json
```

Regla de ciclo de vida en DI: nunca registres como Singleton un servicio que depende de un recurso Scoped como DbContext. Hacerlo genera una 'dependencia cautiva' que causará excepciones en concurrencia y fugas de memoria.

3. Retos adicionales

1. Escribe un método o endpoint auxiliar que mida la diferencia en milisegundos entre una consulta con tracking y otra con AsNoTracking().
2. Usa el método `db.Solicitudes.AsNoTracking().Where(...).ToQueryString()` para inspeccionar e imprimir en consola la sentencia SQL exacta generada por EF Core.
3. Implementa una transacción atómica con `using var tx = await db.Database.BeginTransactionAsync()` para insertar una solicitud con sus detalles en un endpoint POST `/api/solicitudes`.
4. Genera el script T-SQL de la migración de semilla con `dotnet ef migrations script MigracionInicial MigracionSemilla --output migracion-semilla.sql`.
5. Modifica `CambiarEstadoAsync` en `SolicitudService` para persistir la transición en la base de datos y verificar que `FechaModificacion` y `ModificadoPor` se actualizan automáticamente.

4. Entregables

- Base de datos SQL Server con los roles y centros de costo iniciales poblados mediante la migración `MigracionSemilla`.
- `SolicitudesDbContext.cs` con la sobrecarga de `SaveChangesAsync` asignando automáticamente `FechaCreacion` y `CreadoPor` a las entidades auditable.
- `SolicitudService.cs` y `SolicitudesController.cs` actualizados para consumir EF Core de manera incremental con `Include`, `AsNoTracking` y `Select`.
- `Program.cs` configurado con `AddScoped<ISolicitudService, SolicitudService>()` evitando dependencias cautivas.
- Respuestas HTTP en Swagger o cliente REST (.http) demostrando la ejecución de los endpoints `GET /api/solicitudes/{id}`, `GET /api/solicitudes` y `GET /api/solicitudes/resumen`.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 4 · Docente: Edwin Barrientos · New Horizons